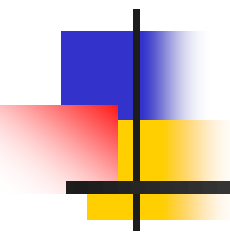
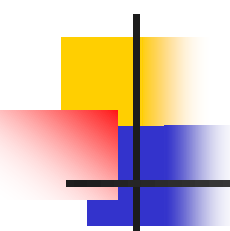


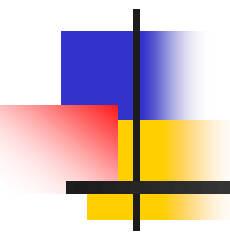
PART 2



RELATIONAL DATABASES



Chapter 8



Relational-Database Design

schema normalization

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Decomposition Using Functional Dependencies
- Functional Dependency Theory
- Algorithms for Functional Dependencies
- (Decomposition Using Multivalued Dependencies) 略
- (More Normal Form) 略
- Database-Design Process
- (Modeling Temporal Data) 略

§ 8.1 Features of Good Relational Design

-Why Normalization Needed ?

- As shown in Fig. 8.0.1, logical DBS design consists of
 - initial relational schema generating (§ 7.6)
 - relational schema normalizing

- A bad DB design, i.e. schema not being normalized well, may result in
 - repetition of information
 - inability to represent certain information

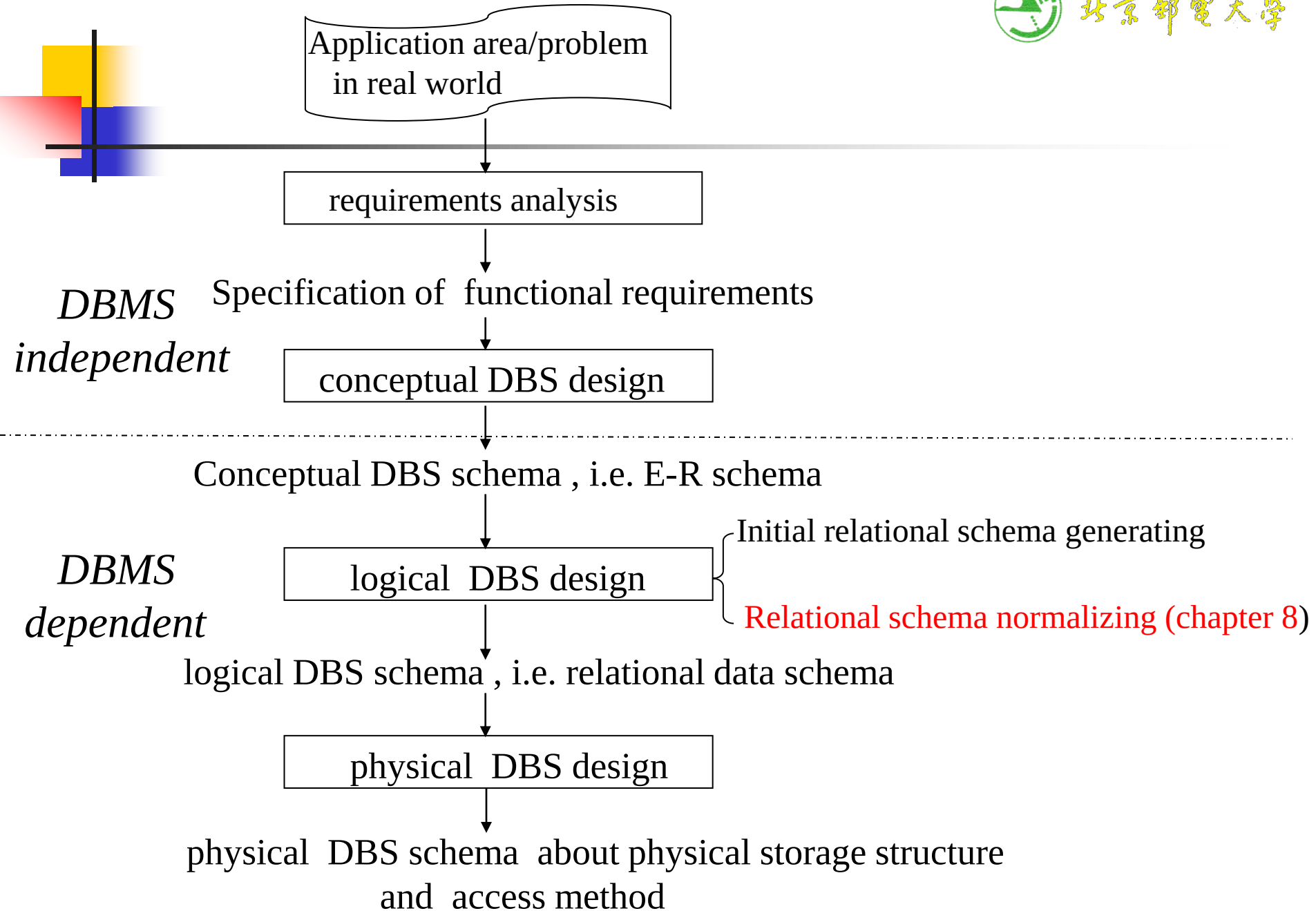


Fig.8.0.1 DBS design

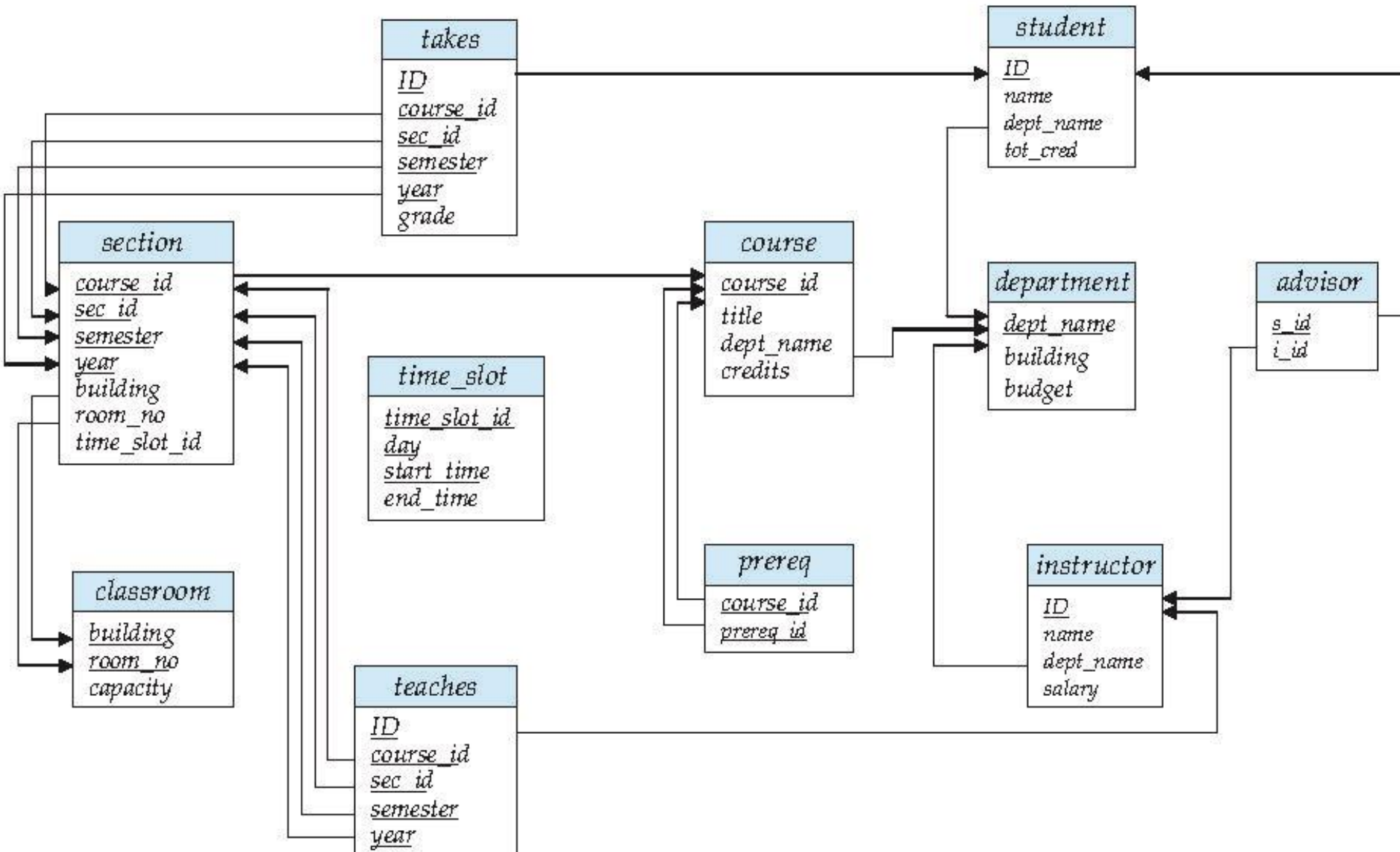


Fig. 8.0.2 Schema Diagram

8.1.1 Combine Schemas

- Suppose we combine *instructor*(*ID*, *name*, *salary*, *dept_name*) and *department*(*dept_name*, *building*, *budget*) into *inst_dept*
 - (No connection to relationship set *inst_dept*)
- Result is possible repetition of information on *department*
 - *many-to-one* mapping cardinality from *instructor* to *department*
- Pitfalls due to information repetition, when
 - *inserting*
 - *deleting*
 - *updating*



ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000
t ₁	201081, Zhang,	60000,	Comp.Sci	Taylor	100000
t ₂	null, null,	null,	Soft.Eng.,	Taylor,	100000

Fig. 8.2



Pitfalls

- *Inserting* problem and *information redundancy*
 - adding a new *instructor* 
 - tuple $t_1 = (201081, \text{Zhang}, 60000, \text{Comp.Sci}, \text{Taylor}, 10000)$ is inserted into *inst_dept*
 - data for the attributes *dept_name*, *building* and *budget* are repeated for the *instructors* in the 4rd, the 7th, the 9th, and the last row that the department “*Comp.Sci*” makes, space is wasted

Pitfalls

- **Deleting** problem
 - e.g. if the department **Comp.Sci** is canceled, all **instructor** in it should then be removed
 - in Fig 8.1, every tuples containing the **Comp.Sci** department should be deleted 
 - the 4rd, the 7th, and the 9th row

Pitfalls

- **Updating** problem and **information redundancy**
 - information redundancy complicates updating, introducing possibility of inconsistency of the **budget** values
 - e.g. change **budget** of the **Comp.Sci** department from 100000 to 120000,
 - in Fig8.2,  every tuples belonging to **Comp.Sci** should be updated

Pitfalls

- Information Representation Problem
 - to represent directly information about a *new-opened* department in which there no exists ***instructor***, a tuple containing *null* values, such as
$$t_2 = (\text{null}, \text{null}, \text{null}, \text{Soft.Eng.}, \textit{Taylor}, 100000)$$
, is inserted ▶
 - *null* values in DB complicate data handling in DBS

- An improved design
 - decompose ***inst_dept***-schema into ***inst*** and ***department***

A Combined Schema Without Repetition

- Consider combining relations
 - *sec_class(sec_id, building, room_number)* and
 - *section(course_id, sec_id, semester, year)*into one relation
 - *section(course_id, sec_id, semester, year, building, room_number)*
- No repetition in this case, why?
 - **one to one mapping** from *section* to *sec_class* cardinality

8.1.2 Smaller Schemas

- Suppose we had started with *inst_dept*. How would we know to split up (**decompose**) it into *instructor* and *department*?
- Write a rule “if there were a schema (*dept_name*, *building*, *budget*), then *dept_name* would be a candidate key”
- Denote as a **functional dependency**:
$$dept_name \rightarrow building, budget$$
- In *inst_dept*, because *dept_name* is not a candidate key, the building and budget of a department may have to be repeated.
 - This indicates the need to decompose *inst_dept*

Smaller Schemas

- Not all decompositions are good. Suppose we decompose *employee*(*ID*, *name*, *street*, *city*, *salary*) into
employee1 (*ID*, *name*)
employee2 (*name*, *street*, *city*, *salary*)
- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition

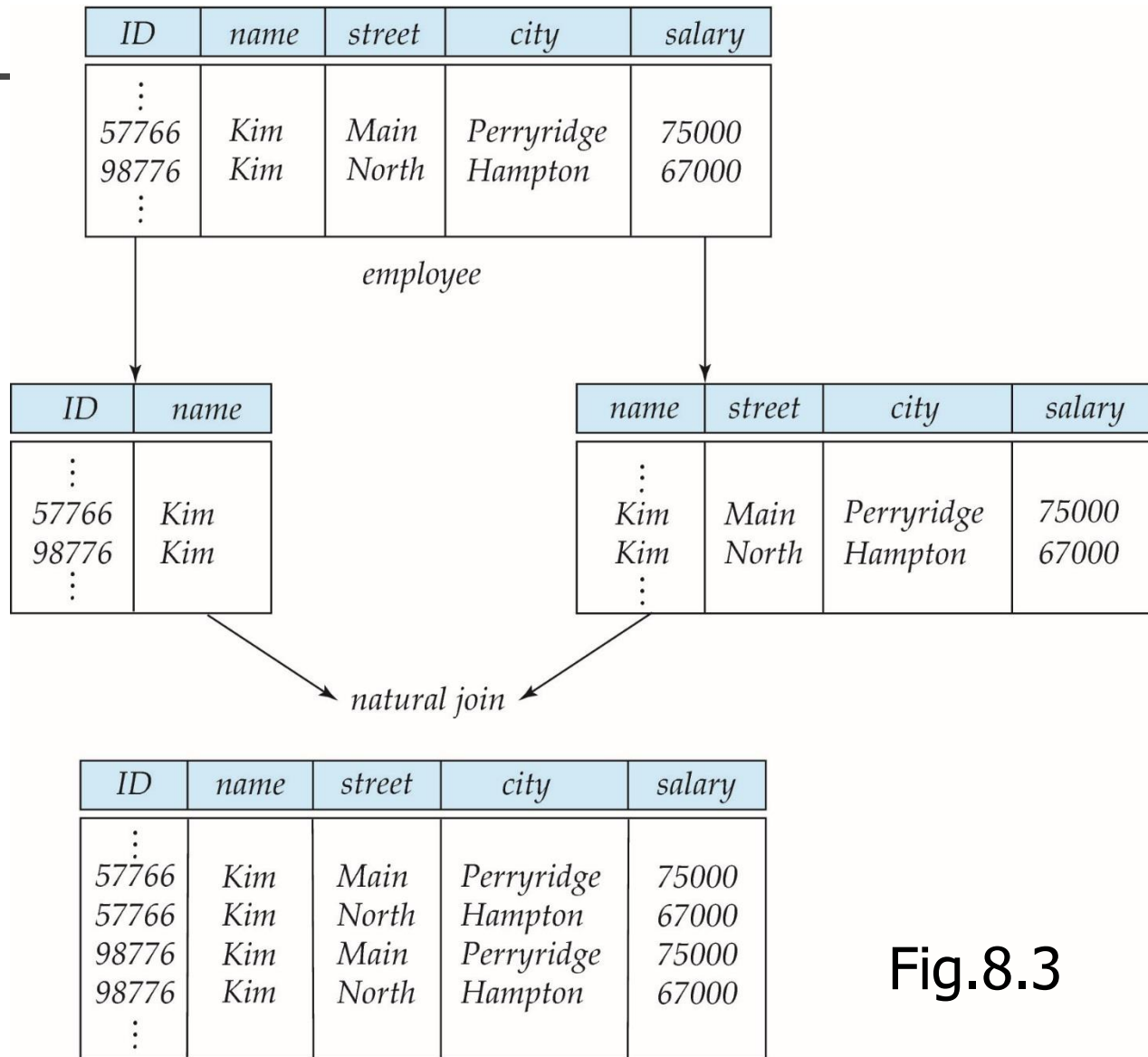


Fig.8.3

Example:

Lossless-Join Decomposition

Lossless join decomposition

- Decomposition of $R = (A, B, C)$

$$R_1 = (A, B) \quad R_2 = (B, C)$$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

Normalization Principles

- 在逻辑DBS设计过程中，将E—R图进行转换，得到面向特定应用领域的初始关系模式集
- 这些初始关系模式集中可能存在多种（作为完整性约束的）关系模式属性间的**数据依赖** (Data Dependencies) 关系
 - **函数依赖** (functional dependencies, FD, § 8.3)
 - **多值依赖** (Multivalued Dependencies, MVD, § 8.6)
 - **连接依赖** (Join Dependencies, JD) 略

Principles of Relation Normalization

(cont.)

- 如果直接根据初始关系模式构造DBS，由于初始关系模式中数据依赖关系的存在，可能会违反DB的完整性约束，导致DBS使用过程中出现如下问题，影响DBS的正确性、性能、效率
 - 数据冗余问题、插入问题、更新问题、删除问题 (pitfalls, § 8.1)

Principles of Relation Normalization

(cont.)

- 因此，对初始关系模式集，需要根据关系规范化理论，在保证关系模式的

- 函数无损连接性 (lossless join), 和/或
- 函数依赖保持性 (dependency preservation)

约束前提下，对关系模式集进行规范化处理——等价变换/
模式分解

- 关系模式规范化主要步骤为
 - 根据函数依赖的Armstrong's 公理系统 (§ 8.4.1)和多值依赖 的公理系统，从初始关系模式集中已知的函数依赖和多值依赖出发，推导出初始关系模式集中所有的函数依赖 (§ 8.4)和多值依赖

Principles of Relation Normalization

(cont.)

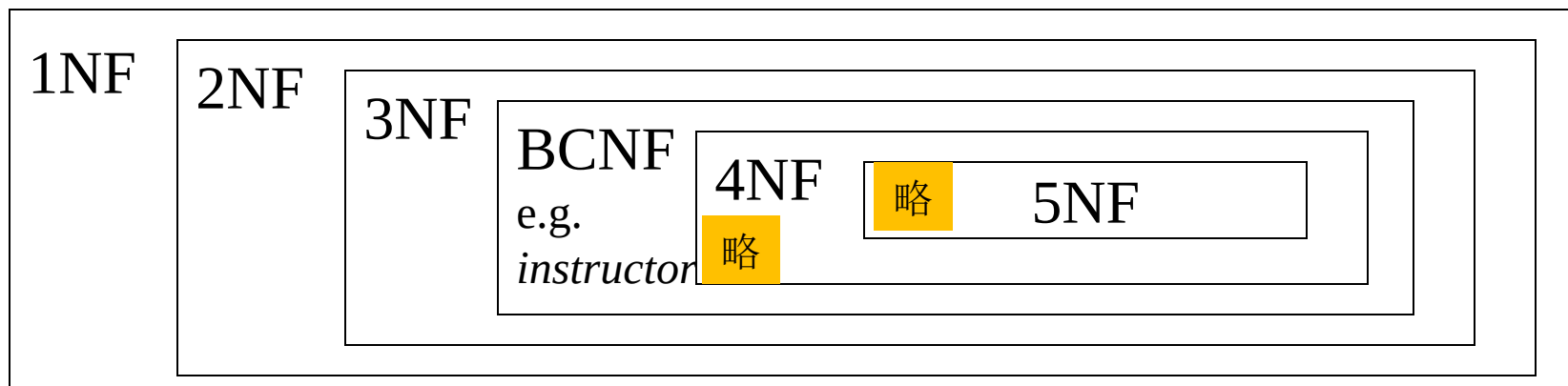
- 对具有函数依赖和多值依赖的初始关系模式集，采用
 - 模式分解算法，
，对其进行（等价）分解和变换，将其转换为各种范式形式，包括：
 - 1NF(§ 8.2) 、 2NF (Exercise 8.16) 、 BCNF(§ 8.3.2) 、 3NF (§ 8.3.4) 、 4NF(§ 8.6.2)
 - ，以消除模式集中的函数依赖和多值依赖带来的负面影响，
保证数据库系统的完整性
- 关系模式规范化处理的基本要求为：
 - 静态关系具有第一范式形式
 - （理论上）动态关系最好具有3NF或BCNF形式

Principles of Relation Normalization

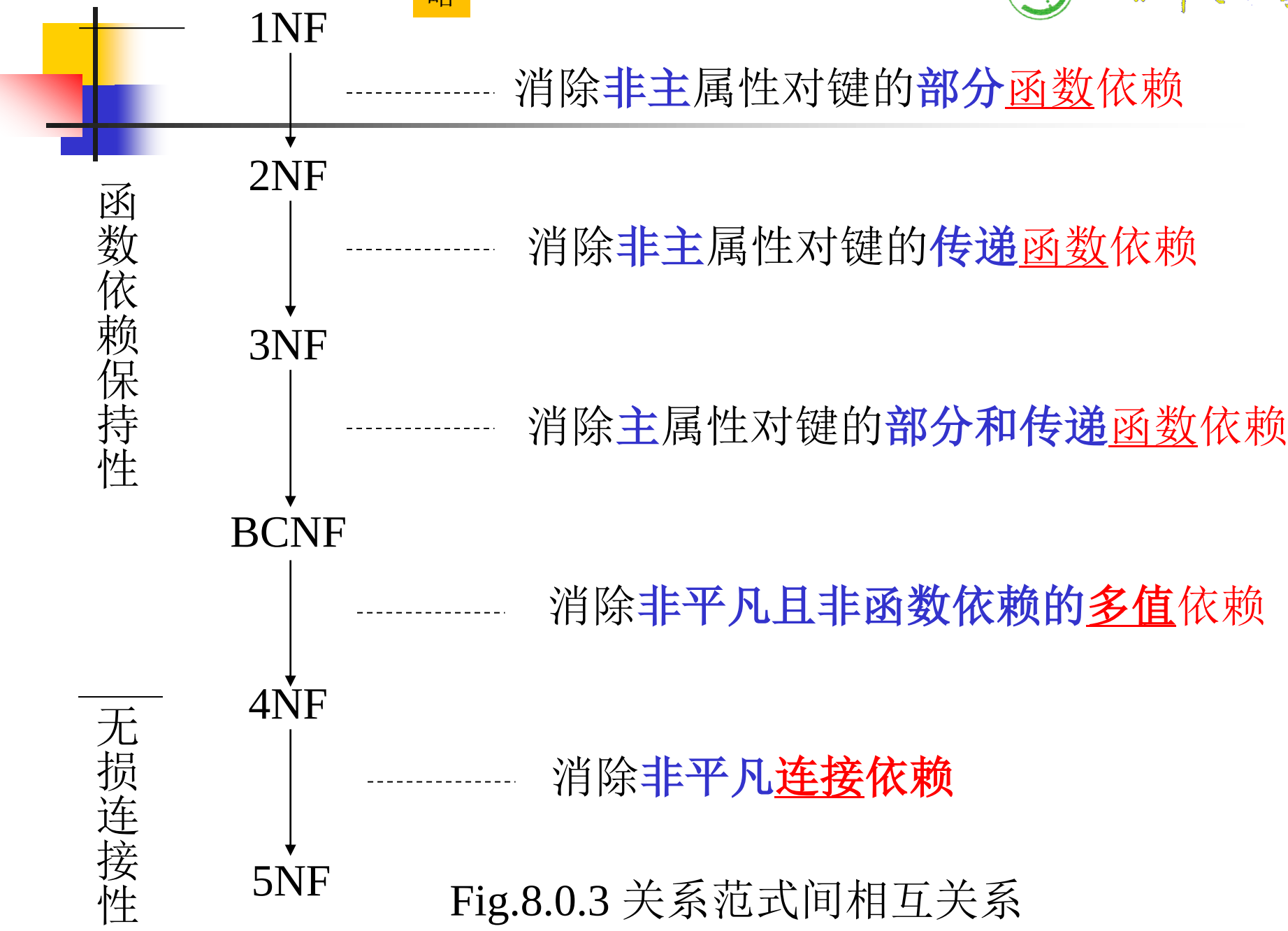
(cont.)

- 3种数据依赖间的关系
 - 函数依赖是特殊的多值依赖
 - 多值依赖又是连接依赖的特例
- 范式1NF、2NF、3NF、BCNF可以看作由符合范式要求的各种关系模式组成的关系模式的集合

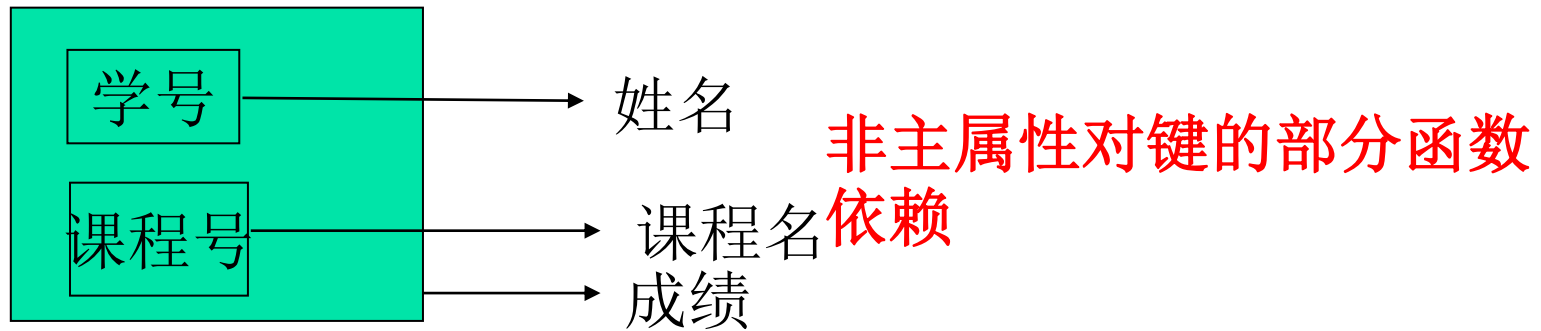
e.g. $1NF = \{ R \mid R \text{满足第一范式的定义} \}$
- 各种范式间的关系，参见Fig.8.0.3



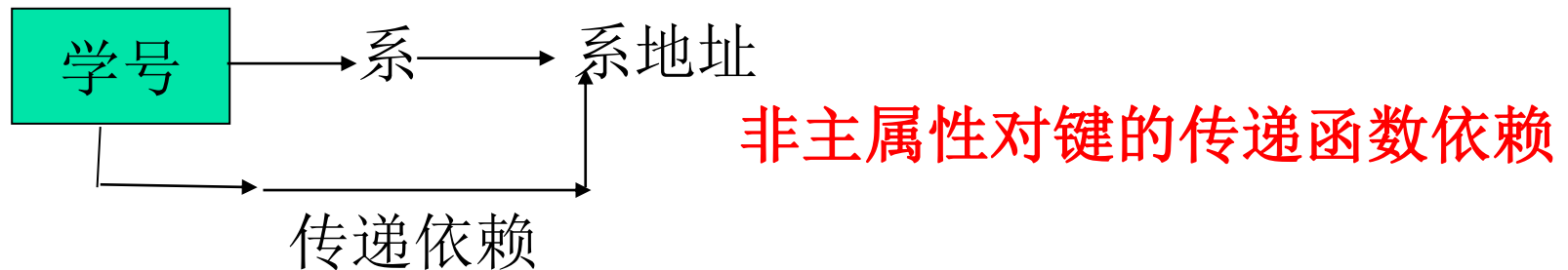
略



■ 1NF: (学号, 姓名, 课程号, 课程名, 成绩)



■ 2NF: (学号, 姓名, 系, 系地址)

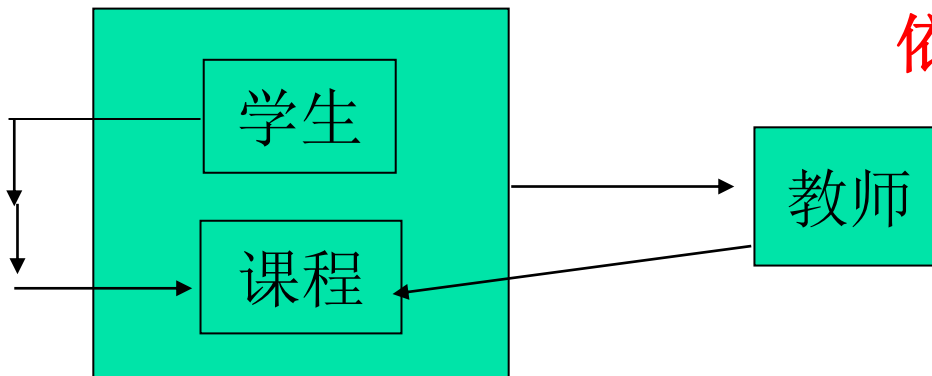


3NF: (学生,教师,课程)

教师只教一门课,每门课有若干教师,某一学生选定某门课,
就对应一个固定的教师

主键(学生,课程)或(学生,教师)

主属性对键的部分和传递函数
依赖



主属性对键的传递依赖:课程依赖学生

Principles of Relation Normalization

(cont.)

- 给定一个关系模式，可以采用规范化算法将其转换为1NF、2NF、3NF、BCNF
- 对连接依赖和第五范式，无相应的模式规范化算法

Goal — Devise a Theory for the Following

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation is in good form
 - the decomposition is a lossless-join decomposition
- Our theory is based on:
 - functional dependencies
 - multivalued dependencies

8.2 Atomic Domains and First Normal Form

- Domain is **atomic** if its elements are considered to be indivisible units
 - Examples of non-atomic domains:
 - Set of names, composite attributes
 - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **first normal form** if the domains of all attributes of R are atomic
- Non-atomic values complicate storage and encourage redundant (repeated) storage of data
 - Example: Set of *accounts* stored with each *customer*, and set of owners stored with each *account*
 - We assume all relations are in first normal form (and revisit this in Chapter 22: Object Based Databases)

First Normal Form (Cont'd)

- Atomicity is actually a property of how the elements of the domain are used.
 - Example: Strings would normally be considered indivisible
 - Suppose that students are given roll numbers which are strings of the form *CS0012* or *EE1127*
 - If the first two characters are extracted to find the department, the domain of roll numbers is not atomic.
 - Doing so is a bad idea: leads to encoding of information in application program rather than in the database.

8.3 Decomposition Using Functional Dependencies

Goal — Devise a Theory for the Following

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation is in good form
 - the decomposition is a lossless-join decomposition
- Our theory is based on:
 - functional dependencies
 - multivalued dependencies

8.3.1 Functional Dependencies

- Constraints on the set of legal relations.
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key*.
- Function: $f: X \rightarrow Y$,
 $x \in X, y \in Y, y = f(x)$
e.g. $y = 2x$
 - for $x_1, x_2 \in X$, if $x_1 = x_2$, then $f(x_1) = f(x_2)$

Functional Dependencies (Cont.)

- Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

- The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A,B)$ with the following instance of r .

1	4
1	5
3	7

- On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.

Functional Dependencies (Cont.)

- **Definition1. Functional dependency(FD)** holds on R

/*函数依赖**FD**在关系模式**R**上成立/保持*/

Let R be a relation schema, and

$$\alpha \subseteq R, \beta \subseteq R$$

, the functional dependency $\alpha \rightarrow \beta$ holds on schema R

if and only if

for **any** legal relation (or *relation instance*) $r(R)$, whenever **any two tuples** t_i and t_j in r **agree on** the attributes α , they also *agree on* the attributes β , that is,

$$t_i[\alpha] = t_j[\alpha] \Rightarrow t_i[\beta] = t_j[\beta]$$

Functional Dependencies (Cont.)

- **Keys** in relational schema can be defined in terms of FD
 - K is a superkey for relation schema R, if and only if $K \rightarrow R$
 - K is a candidate key for R, if and only if
 - $K \rightarrow R$, and
 - for no $\alpha \subset K$, $\alpha \rightarrow R$

Functional Dependencies (Cont.)

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys.
- Consider the schema:

inst_dept (ID, name, salary, dept_name, building, budget).

We expect these functional dependencies to hold:

dept_name* → *building

and ***ID* → *building***

but would not expect the following to hold:

dept_name* → *salary

Functional Dependencies (Cont.)

- **Definition2.** (A particular *relation instance*) $r(R)$ satisfy FD, or FD is satisfied by $r(R)$

/*关系 $r(R)$ 满足函数依赖集FD, FD 被 r 满足*/

Given $FD = \{\alpha \rightarrow \beta\}$ holding on R , and a relation (*instance*) $r(R)$ on R ,

- r is said to satisfy FD, if r is *legal* under functional dependency set FD
- note:
 - r is *legal* under $FD = \{\alpha \rightarrow \beta\}$ means
 - for $t_i, t_j \in r(R)$, if $t_i[\alpha] = t_j[\alpha]$, then $t_i[\beta] = t_j[\beta]$
- FD requires that the values for a certain set of attributes determines uniquely the value for another set of attributes

Example One

- Given relation $r(R)$ shown below, which FD is satisfied by r
 - A. $A \rightarrow B$ B. $AC \rightarrow B$ C. $BC \rightarrow A$ D. $B \rightarrow C$

	A	B	C
t1	1	4	2
t2	3	5	6
t3	3	4	6
t4	7	3	8
t5	9	1	0

Fig. 7.0.4

Example One (cont.)

- $t_2[A]=t_3[A]=3, t_2[B]=5 \neq t_3[B]=4,$

$A \rightarrow B$ is not satisfied

- $t_2[AC]=t_3[AC]=36, t_2[B]=5 \neq t_3[B]=4,$

$AC \rightarrow B$ is not satisfied

- $BC \rightarrow A$ is satisfied

- $t_1[B]=t_3[B]=4, t_1[C]=2 \neq t_3[C]=6,$

$B \rightarrow C$ is not satisfied

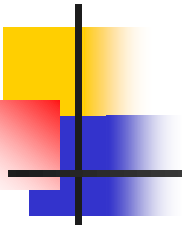
Example Two

- Consider the schema $R = (\text{employee_ID}, \text{date}, \text{turnover per-day}, \text{department_name}, \text{manager})$ that describes the information about the turnover per-day (日营业额) for each employee everyday, the department that each employee works at, and the manager of the department the employee works at.
 - it is assumed that
 - at every *day*, each *employee* has **only one** *turnover per-day*
 - each *employee* works at **only one** department
 - each *department* is managed by **only one** *manager*
- According to the descriptions mentioned above, list all the functional dependencies that hold on R
- Q&A: *How about the E-R diagram for this example?*

Example Two (cont.)

- Answer

- $F = \{ \text{employee_ID, } \mathbf{date} \rightarrow \text{turnover per-day,}$
 $\text{employee_ID} \rightarrow \text{department_name,}$
 $\text{department_name} \rightarrow \text{manager}$
 $\}$



- While E-R diagrams describe the objects in practice and the associations among them, functional dependencies (FDs) illustrate the relationships among the features/attributes of the objects (i.e. entities or relationships in E-R diagrams)
- How to guarantee the FD in DBS?
 - integrity mechanisms in DBMS, i.e. keys, assertions, triggers

Use of Functional Dependencies

- We use functional dependencies to:
 - test relations to see if they are legal under a given set of functional dependencies.
 - If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F .
 - specify constraints on the set of legal relations
 - We say that F **holds on** R if all legal relations on R satisfy the set of functional dependencies F .
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
 - For example, a specific instance of *instructor* may, by chance, satisfy
$$name \rightarrow ID.$$

FD holds on R vs FD is satisfied by $r(R)$

- FD holds on R

- 定义在 R 的属性间的语义约束，或 R 的属性间体现出的语义约束
- 从设计角度， R 应满足的约束

- FD is satisfied by $r(R)$

- 根据 R 构造的实际数据 $r(R)$ 是否满足语义约束FD

FD holds on R vs FD is satisfied by $r(R)$

- For a schema R , there may be more than one relation instance $r(R)$, i.e. $r_1(R)$, $r_2(R)$, $r_3(R)$, ..., $r_m(R)$, defined on R
 - e.g. consider $R = (A, B, C, D)$, and with respect to the instances $r_1(R)$ and $r_2(R)$ in Fig.7.5
- Relation $r(R)$ satisfies $\alpha \rightarrow \beta$ vs. $\alpha \rightarrow \beta$ holds on schema R
 - if $\alpha \rightarrow \beta$ holds on R , then every **legal** $r(R)$ satisfies this R
 - but for schema R and $\alpha \rightarrow \beta$, if only some $r_i(R)$ satisfies R , $\alpha \rightarrow \beta$ may not hold on R .
 - e.g. in Fig.7.5, $A \rightarrow C$ and $AB \rightarrow D$ are satisfied by $r_1(R)$, but $A \rightarrow C$ is not satisfied by $r_2(R)$, so $A \rightarrow C$ does not hold on R

$FD_1 = \{A \rightarrow C, AB \rightarrow D\}$
 , satisfied by r_1

$FD_2 = \{AB \rightarrow D\}$
 , satisfied by r_2

r_1

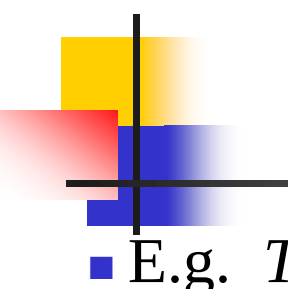
	A	B	C	D
	a_1	b_1	c_1	d_1
t2	a_1	b_2	c_1	d_2
	a_2	b_2	$c_2 \leftarrow$	d_2
	a_2	b_3	c_2	d_3
	a_3	b_3	$c_2 \leftarrow$	d_4

r_2

	A	B	C	D
	a_1	b_1	c_1	d_1
t2	a_1	b_2	$c_1 \rightarrow$	d_2
	a_2	b_2	c_2	d_2
	a_2	b_3	c_2	d_3
	a_3	b_3	c_2	d_4
t6	a_1	b_4	$c_2 \leftarrow$	d_5

$A \rightarrow C$ does not hold on R , because of t6

Fig.8.0.5 instance r_1 and r_2 defined on schema R



■ E.g. *True or false ?*

- for a relation $r(R)$ defined on schema R , if r satisfies functional dependency $FD = \{\alpha \rightarrow \beta\}$, then FD holds on schema R
- false

Functional Dependencies (Cont.)

- A functional dependency is **trivial** if it is satisfied by all instances of a relation
 - Example:
 - $ID, name \rightarrow ID$
 - $name \rightarrow name$
 - In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

VII-2-1 Basic Concepts (cont.)

- Def. Transitive dependency (传递函数依赖)
 - a functional dependency $\alpha \rightarrow \gamma$ is *transitive* if:
 $\alpha \rightarrow \beta, (\beta \not\subseteq \alpha), \beta \twoheadrightarrow \alpha, \beta \rightarrow \gamma$
; and γ is called transitive dependent on α
- E.g. *Student(sno, sname, address, depart)*
 - for *transitive* dependency
 $sno \rightarrow address$
, there are
 $sno \rightarrow sname, sname \rightarrow address$

VII-2-1 Basic Concepts (cont.)

- **Def.** Partial dependency (部分函数依赖)
 - a functional dependency $\alpha \rightarrow \beta$ is *partial* if there is a proper subset γ of α , i.e. $\gamma \subset \alpha$, such that
$$\gamma \rightarrow \beta$$
; and β is partially dependent on α
 - /* α 非最小化/冗余
- E.g. Student(*sno*, *sname*, *address*, *depart*)
 - for *partial* dependency
$$(sno, sname) \rightarrow address$$
, there are $sname \rightarrow address, sno \rightarrow address$

Closure of a Set of Functional Dependencies

- Given a set F of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - For example: If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied by F is the **closure** of F , denoted as F^+ .
- refer to 8.4.1

8.3.2 Boyce-Codd Normal Form

A relation schema R is in BCNF with respect to a set F of functional dependencies if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
- α is a superkey for R

Example schema *not* in BCNF:

instr_dept (ID, *name*, *salary*, *dept_name*, *building*, *budget*)

because *dept_name* $\rightarrow$ *building*, *budget*

holds on *instr_dept*, but *dept_name* is not a superkey

BCNF (cont.)

消除主属性对键的 **部分和传递函数依赖**

3NF $\longrightarrow$ BCNF

- BCNF性质
 - 不存在属性（主属性，非主属性）对候选键的传递和部分依赖
- If R is in BCNF, R is also in 3NF
- 以FD作为模式规范化程度度量，BCNF达到了最高规范化程度。
- 如果DBS中所有模式都属于BCNF，则消除了插入和删除异常

Decomposing a Schema into BCNF

- Suppose we have a schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF.

We decompose R into:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

- In our example,

- $\alpha = dept_name$
- $\beta = building, budget$

and $inst_dept$ is replaced by

- $(\alpha \cup \beta) = (dept_name, building, budget)$
- $(R - (\beta - \alpha)) = (ID, name, salary, dept_name)$

8.3.3 BCNF and Dependency Preservation

- Constraints, including functional dependencies, are costly to check in practice unless they pertain to only one relation
- If it is sufficient to test only those dependencies on each individual relation of a decomposition in order to ensure that *all* functional dependencies hold, then that decomposition is *dependency preserving*.
- Because it is not always possible to achieve both BCNF and dependency preservation, we consider a weaker normal form, known as *third normal form* (3NF).

8.3.4 Third Normal Form

- **Def 1:** A relation schema R is in **third normal form (3NF)** if for all:

$$\alpha \rightarrow \beta \text{ in } F^+$$

at least one of the following holds:

- $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \in \alpha$)
- α is a superkey for R
- Each attribute A in $\beta - \alpha$ is contained in a candidate key for R .

E.g. $B \rightarrow ABC$,
 $(ABC) - (B) = AC$

(**NOTE:** each attribute may be in a different candidate key)

- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold).
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation (will see why later).

VII-3-3 Third Normal Form (cont.)

Def.2. 如果关系模式 R 是2NF，而且每一个非主属性都不传递依赖于 R 的任何候选键，则 R 是3NF

消除 **非主**属性对键的 **传递**函数依赖

2NF $\longrightarrow$ 3NF

- If R is in 3NF,
- 3NF 性质
 - R is also in 2NF
 - 不存在非主属性对候选键的 **部分**和 **传递**依赖，i.e.每一个非主属性都不传递依赖于 R 的任何候选键

Example 1 about 3NF

- Scheme SSS (S# , SD , SL)

$F_{SSS} = \{S\# \rightarrow SD, SD \rightarrow SL, S\# \rightarrow SL\}$ 闭包

- On basis of 3NF definition1, SSS is not **in** 3NF, because

- for $SD \rightarrow SL$, RHS SL in $\{SL\} - \{SD\} = \{SL\}$ is *not in* candidate key $\{S\# \}$

(/*each attribute A in $\beta - \alpha$ is contained in a candidate key for R)

- On basis of 3NF definition2, SSS is not **in** 3NF, because

- for $S\# \rightarrow SL$, RHS SL is non-primacy attribute, and is **transitive dependent** on candidate key $\{S\# \}$

(**非主属性对键的传递函数依赖**)



Appendix Second Normal Form (P372 8.17)

- **Def.** A schema R is in second normal form (2NF) if
 - R is in 1NF, and
 - each attribute A in R meets *one of* the following criteria
 - it appears in a candidate key, *that is* A is a *prime attribute*
 - it is not *partially* dependent on a candidate key
/* A is completely dependent on a candidate key */

VII-3-2 Second Normal Form (cont.)

1NF 消除 **非主**属性对候选键的 **部分**函数依赖 2NF

■ 2NF 性质

- 不存在非主属性对候选键的 **部分**依赖
- 每一个 **非主属性**都**完全**依赖于R的**候选**键

2NF Example

- E.g. $SLC(\underline{S\#}, SD, SL, \underline{C\#}, G)$
 - $(S\#, C\#) \rightarrow G, \quad SD \rightarrow SL,$
 $S\# \rightarrow SD, \quad (S\#, C\#) \rightarrow SD,$
 $S\# \rightarrow SL, \quad (S\#, C\#) \rightarrow SL\}$
- SLC is not in 2NF, because
 - for non-primary attribute SD , there exists $S\# \rightarrow SD$, so SD is partially dependent on the key $(S\#, C\#)$
 - for non-primary attribute SL , there exists $S\# \rightarrow SL$, so SL is partially dependent on the key $(S\#, C\#)$

2NF Example (cont.)

- Decompose $SLC(\underline{S\#}, SD, SL, \underline{C\#}, G)$ into
 $SC(\underline{S\#}, \underline{C\#}, G), \quad SSS(\underline{S\#}, SD, SL)$
 - $F_{SC} = \{ (S\#, C\#) \rightarrow G \}$
 $F_{SSS} = \{ S\# \rightarrow SD, SD \rightarrow SL, S\# \rightarrow SL \}$
 - schema SC and SSS are in 2NF

Goals of Normalization

- Let R be a relation scheme with a set F of functional dependencies.
- Decide whether a relation scheme R is in “good” form.
- In the case that a relation scheme R is not in “good” form, decompose it into a set of relation scheme $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation scheme is in good form
 - the decomposition is a lossless-join decomposition
 - Preferably, the decomposition should be dependency preserving.

How good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized

- Consider a relation

inst_info (*ID*, *child_name*, *phone*)

- where an instructor may have more than one phone and can have multiple children

<i>ID</i>	<i>child_name</i>	<i>phone</i>
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info

How good is BCNF? (Cont.)

- There are no non-trivial functional dependencies and therefore the relation is in BCNF
- Insertion anomalies – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples

(99999, David, 981-992-3443)

(99999, William, 981-992-3443)

How good is BCNF? (Cont.)

- Therefore, it is better to decompose *inst_info* into:

<i>inst_child</i>	<i>ID</i>	<i>child_name</i>
	99999	David
	99999	David
	99999	William
	99999	Willian

<i>inst_phone</i>	<i>ID</i>	<i>phone</i>
	99999	512-555-1234
	99999	512-555-4321
	99999	512-555-1234
	99999	512-555-4321

This suggests the need for higher normal forms, such as Fourth Normal Form (4NF), which we shall see later.

8.4 Functional-Dependency Theory

- We now consider the formal theory that tells us which functional dependencies are implied logically by a given set of functional dependencies.
- We then develop algorithms to generate lossless decompositions into BCNF and 3NF
- We then develop algorithms to test if a decomposition is dependency-preserving

8.4.1 Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain other functional dependencies that are *logically implied* by F .
 - e.g.
If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied by F is the **closure** of F .
- We denote the *closure* of F by F^+ .

$F = \{A \rightarrow B, B \rightarrow C\};$

$f: A \rightarrow C$ is logically implied by F

	A	B	C
t1	1	4	2
t2	3	5	6
t3	4	4	2
t4	7	3	8
t5	9	1	0

Fig. 8.0.5

- **Def.** Given a schema R , a functional dependency f on R is *logically implied* by a set of **FD** F on R , if
 - *every* instance $r(R)$ that satisfies F also satisfies f
 - e.g. Fig. 8.0.5
- **Def.** Given a set F of functional dependencies, the *closure* of F , denoted as F^+
 - $F^+ = \{ f \mid f \text{ is logically implied by } F \}$
 - e.g. in Fig. 8.0.5, $\{A \rightarrow B, B \rightarrow C\}^+$
 $= \{A \rightarrow B, B \rightarrow C, A \rightarrow C, \dots\}$

Closure of a Set of Functional Dependencies

- We can find F^+ , the closure of F , by repeatedly applying

Armstrong's Axioms:

- if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity** 自反律)
- if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ (**augmentation** 增广律)
- if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity** 传递律)
- These rules are
 - **Sound** (正确有效的) (generate only functional dependencies that actually hold), and
 - **Complete** (完备的) (generate all functional dependencies that hold).

Closure of Functional Dependencies (Cont.)

- Additional rules:
 - If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds (**union**合并律)
 - If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds (**decomposition**分解律)
 - If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds (**pseudotransitivity**伪传递律)

The above rules can be inferred from Armstrong's axioms.

An Example of Functional Dependency

■ Question

Which rule about functional dependencies shown below is right

- A. if $\alpha \rightarrow \beta$ then $\beta \rightarrow \alpha$
- B if $A \rightarrow C$, $BC \rightarrow D$ then $AB \rightarrow D$
- C. if $AB \rightarrow C$ then $B \rightarrow C$
- D if $\alpha \subseteq \beta$, then $\alpha \rightarrow \beta$

■ Answer: B

Example

■ $R = (A, B, C, G, H, I)$

$F = \{ A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H\}$

■ some members of F^+

■ $A \rightarrow H$

■ by transitivity from $A \rightarrow B$ and $B \rightarrow H$

■ $AG \rightarrow I$

■ by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$

Example (cont.)

■ $R = (A, B, C, G, H, I)$

$F = \{ A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H\}$

■ some members of F^+

■ $CG \rightarrow HI$

- by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
and then transitivity

Procedure for Computing F^+ 略

- To compute the closure of a set of functional dependencies F :

$$F^+ = F$$

repeat

for each functional dependency f in F^+

 apply reflexivity and augmentation rules on f

 add the resulting functional dependencies to F^+

for each pair of functional dependencies f_1 and f_2 in F^+

if f_1 and f_2 can be combined using transitivity

then add the resulting functional dependency to

F^+

until F^+ does not change any further

NOTE: We shall see an alternative procedure for this task later

8.4.2 Closure of Attribute Sets

- **Def.** An attribute B is functionally determined by α if $\alpha \rightarrow B$
- **Def.** Given a set of attributes α , the *closure of α under F* , denoted by α^+ , is
$$\{\beta \mid \beta \text{ is } \textit{functionally determined} \text{ by } \alpha \text{ under } F\}$$

■

Closure of Attributes (cont.)

Input: α, F

Output: α^+

$result := \alpha;$

while (changes to $result$) **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

begin

if $\beta \subseteq result$ /* $result = (\beta, \dots)$

then $result := result \cup \gamma$

end

Fig.7.9 An *efficient* algorithm to compute α^+ under F

An Example

■ $R = (A, B, C, G, H, I)$

■ $F = \{A \rightarrow B,$
 $A \rightarrow C,$
 $CG \rightarrow H, CG \rightarrow I$
 $B \rightarrow H \quad \quad \quad \}$

■ Computing $(AG)^+$

1. $result = \textcolor{red}{A}G$ /* or denoted as $\{A, G\}$

2. $result = A\textcolor{red}{B}\textcolor{red}{C}\textcolor{red}{G}$ /* $A \rightarrow C, A \rightarrow B$

3. $result = ABCG\textcolor{red}{H}$ /* $CG \rightarrow H$

4. $result = ABCGHI$ /* $CG \rightarrow I$
 /* or $\{A, B, C, G, H, I\}$

An Example (cont.)

- $(AG)^+ = R$, AG is a *superkey* of R
- Is AG a candidate key?
 - step1. is AG a super key?
 - does $AG \rightarrow R?$ == Is $(AG)^+ = R$
 - *yes*
 - step2. is any subset of AG a superkey?
 - does $A \rightarrow R?$ == is $(A)^+ = R?$, *no*
 - does $G \rightarrow R?$ == is $(G)^+ = R?$, *no*
 - so, AG is a candidate key

Uses of Attribute Closure

■ Usage-I. Testing for superkey

To test whether α is a superkey of R **under** F , i.e. whether $\alpha \rightarrow R$, we check if

$$R = \alpha^+$$

■ Usage-II. Testing functional dependencies

To determine whether or not $\alpha \rightarrow \beta$ holds on R under F , we check if

$$\beta \subseteq \alpha^+$$

That is, we compute α^+ by using attribute closure, and then check if it contains β .

Is a simple and cheap test, and very useful

■ Usage-III. Computing closure F^+

for each $\gamma \subseteq R$, compute $\gamma^+ = \{S\}$ under F ;

for each $S \subseteq \gamma^+$, output $\gamma \rightarrow S$ as a functional dependency in F^+

Closure of Attributes (cont.)

- **Def.** For functional dependencies F and G , if

$$F^+ = G^+$$

then F and G are equivalent

- E.g. $F = \{A \rightarrow B, B \rightarrow C\}$ is equivalent to
 $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

8.4.3 Canonical Cover

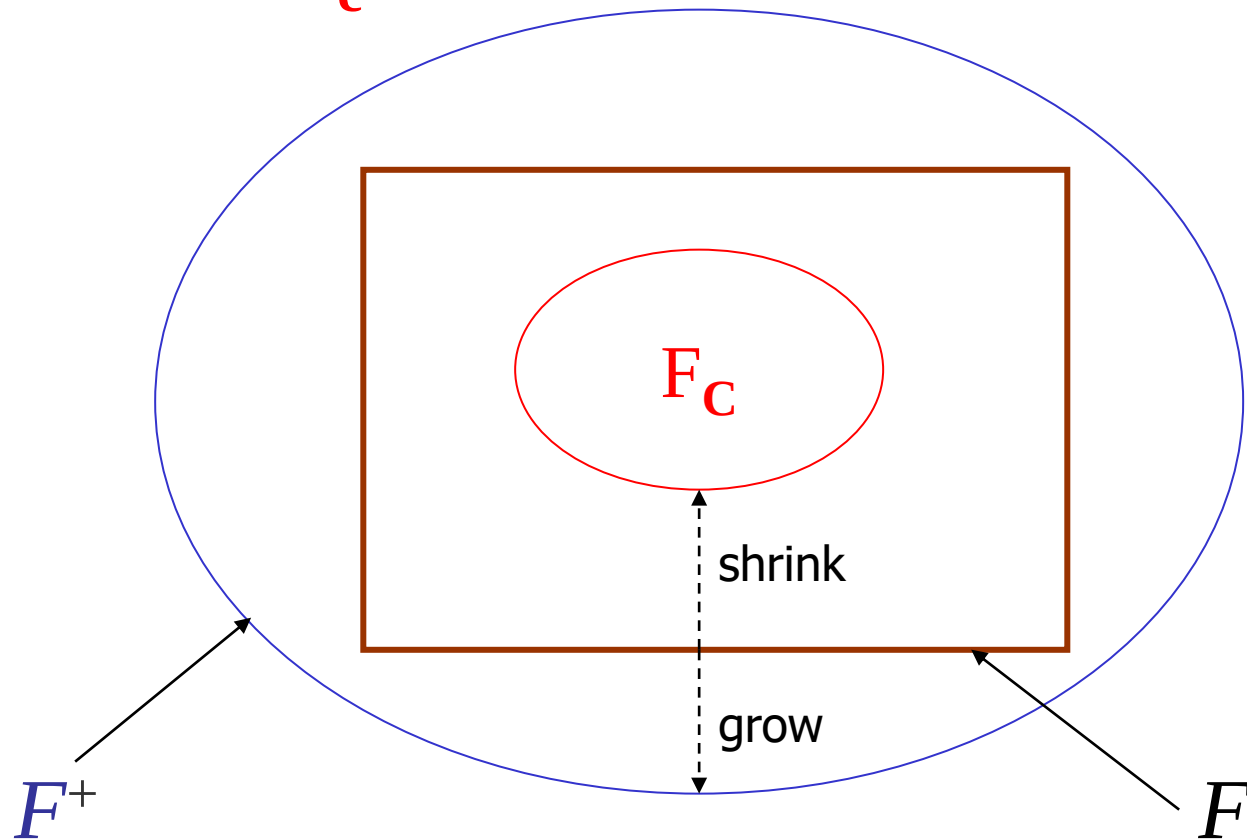
- checking *all* $\alpha \rightarrow \beta$ in F is time-consuming
 - F may have *redundant* dependencies that can be inferred from the others
- Sets of functional dependencies may have redundant dependencies that can be inferred from the others
 - E.g. $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
 - Parts of a functional dependency may be redundant
 - E.g. on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$
 - E.g. on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$

Canonical Cover

- It is desirable to test a “*minimal*” set of functional dependencies *equivalent* to F
- Intuitively, a *canonical cover* (正则覆盖) of F , denoted as F_c , is
 - a “*minimal*” set of functional dependencies *equivalent* to F
 - without redundant functional dependencies or attributes
 - $F^+ = F_c^+$

F , F^+ , and F_C

$$F^+ = \mathbf{F}_c^+$$



Extraneous Attributes

- **Def.** Consider a set F of functional dependency and $\alpha \rightarrow \beta$ in F ,
 - attribute A is **extraneous** (无关的) in α , if
 - $A \in \alpha$, and
 - F implies/*is equivalent to* $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
that is, $F^+ \supseteq (F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
 - attribute A is **extraneous in β** , if
 - $A \in \beta$, and
 - the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ implies F
that is, $((F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\})^+ \supseteq F$

Extraneous Attributes

- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
 - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$ (I.e. the result of dropping B from $AB \rightarrow C$).

- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
 - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C

Testing if an Attribute is Extraneous

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F ,
- to test if attribute $A \in \alpha$ is extraneous

$$(\alpha - A) \rightarrow \beta \text{ under } F?$$

1. compute $(\alpha - A)^+$ using the dependencies in F
2. check $\beta \in (\alpha - A)^+ ?$

if it does, then $(\alpha - A) \rightarrow \beta$ holds, A is extraneous

Testing if an Attribute is Extraneous

- to test if attribute $A \in \beta$ is extraneous in β
 $\alpha \rightarrow A$ under F'
 1. compute α^+ using only the dependencies in
 $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$,
 2. check $A \in \alpha^+$? if it does, $\alpha \rightarrow A$ holds, A is extraneous

Canonical Cover

- A **canonical cover** for F is a set of dependencies F_c such that
 - F logically implies all dependencies in F_c , and
 - F_c logically implies all dependencies in F , and
 - No functional dependency in F_c contains an extraneous attribute, and
 - Each left side of functional dependency in F_c is unique, that is, there are no two dependencies $\alpha_1 \rightarrow \beta_1, \alpha_2 \rightarrow \beta_2$ in F_c , such that $\alpha_1 = \alpha_2$

Canonical Cover

■ To compute a canonical cover for F :

repeat

1. Use the **union** rule to replace any dependencies in F
 $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$

2. Find a functional dependency $\alpha \rightarrow \beta$ with an
extraneous attribute either in α or in β

if an extraneous attribute is found, **delete** it from $\alpha \rightarrow \beta$

until F_c does not change

Canonical Cover (cont.)

- Note: ***Union*** rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied
- Note:
for a set F of functional dependencies, there may be ***several*** F_c ,

Example of Canonical Cover

- Computing a Canonical Cover for

$R = (A, B, C)$

$F1 = \{ A \rightarrow BC ,$

$B \rightarrow C,$

$A \rightarrow B,$

$AB \rightarrow C\}$

- applying the **Union** rule to combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$, F_c becomes

$F = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

- A is extraneous in $AB \rightarrow C$, because

- F logically implies $\{A \rightarrow BC, B \rightarrow C\} \cup \{B \rightarrow C\}$; or

Example of Canonical Cover (cont.)

- $\{AB-A\}^+ = \{B\}^+$ under F is $\{BC\}$, and contains C ,
- F_c is now $\{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$, because
 - $\{A \rightarrow B, B \rightarrow C\}$ logically implies $\{A \rightarrow BC, B \rightarrow C\}$, or
 - $(A)^+$ under $\{A \rightarrow B, B \rightarrow C\}$ is $\{BC\}$, and contains C
- F_c is:
$$A \rightarrow B$$
$$B \rightarrow C$$

8.4.4 Lossless-join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+ :
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

$$r(R) = r_1 \bowtie r_2 \bowtie r_3 \text{ ?}$$

/*无损连接分解中， R 所对应的 r 被分成若干垂直片段 $\Pi_{R_i}(r)$ ，各垂直片段通过自然连接可恢复 r 中的数据，保证了数据的完整性/分解可恢复性

■ $R(A_1, A_2, \dots, A_i, \dots, A_n), R = R_1 \cup R_2 \cup R_3$

$r(R)$

A_1	A_2		A_i		A_{n-1}	A_n
*	*	*	*	*	*	*
*	*	*	*	*	*	*
*	*	*	*	*	*	*
*	*	*	*	*	*	*

$\underbrace{\hspace{15em}}_{r_1 = \Pi_{R_1}(r)}$

$\underbrace{\hspace{15em}}_{r_2 = \Pi_{R_2}(r)}$

$\underbrace{\hspace{15em}}_{r_3 = \Pi_{R_3}(r)}$

$\dots$

Fig. Decomposition of R and $r(R)$

Lossless Decomposition (cont.)

- Def. Lossy decomposition /*有损连接分解*/

$$r \neq \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \dots \bowtie \Pi_{R_n}(r)$$

- also known as *lossy-join* (有损连接分解) decomposition
- Lossy decompositions may result in *information loss*

Example

- $R = (A, B, C)$

$$F = \{A \rightarrow B, B \rightarrow C\}$$

- Can be decomposed in two different ways

- $R_1 = (A, B), R_2 = (B, C)$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$

- Dependency preserving

- $R_1 = (A, B), R_2 = (A, C)$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$

- Not dependency preserving

(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

8.4.5 Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i .
 - A decomposition is **dependency preserving**, if
$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$
 - If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive.

Dependency Preserving (cont.)

- **Def.** For a schema $\mathbf{R}$, F holds on $\mathbf{R}$, and the decomposition $\{R_1, R_2, \dots, R_n\}$ of $\mathbf{R}$,

the **restriction of F to R_i** , denoted as F_i is defined as

$$F_i = \{\alpha \rightarrow \beta \mid \alpha \rightarrow \beta \in F^+ \text{ AND } \alpha\beta \subseteq R_i\}$$

- i.e. the set of dependencies in F^+ that include only attributes in R_i /*限制、投影
- e.g. **example** in the *next* slide

Dependency Preserving

- $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow D, B \rightarrow D\}$ on R
 - $F^+ = F \cup \{A \rightarrow C\}$
 - lossless decomposition:
 $R_1(A, B, C)$, $F_1 = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ on R_1 ,
 $R_2(B, D)$, $F_2 = \{B \rightarrow D\}$ on R_2
 - note: $A \rightarrow D$ is lost on R_1 and R_2

Testing for Dependency Preservation

- for $\mathbf{F} = \{\alpha \rightarrow \beta\}$, apply the following procedure for each $\alpha \rightarrow \beta$

result = α

while (changes to *result*) do

for each R_i in the decomposition

$t = (\text{result} \cap R_i)^+ \cap R_i$ (with respect to $\mathbf{F}$)

$\text{result} = \text{result} \cup t$

/*利用只包含在
各个子模式 R_i 中的
中间结果 result 去
推导

- if **result** contains all attributes in β , then the functional dependency $\alpha \rightarrow \beta$ is preserved
- The decomposition is preserved, if and only if **all** $\alpha \rightarrow \beta$ in $\mathbf{F}$ are preserved.

Testing for Dependency Preservation

- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute F^+ and $(F_1 \cup F_2 \cup \dots \cup F_n)^+$

Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B$
 $B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF
- Decomposition $R_1 = (A, B), R_2 = (B, C)$
 - R_1 and R_2 in BCNF
 - Lossless-join decomposition
 - Dependency preserving

Example

- Student(*sno*, *dept*, *head*),
 $F = \{sno \rightarrow dept, dept \rightarrow head\}, \quad F^+ = F \cup \{sno \rightarrow head\} \cup \{\dots\}$
- Decomposition 1
 $R_1(sno, dept), \quad F_1 = \{sno \rightarrow dept\}$
 $R_2(sno, head), \quad F_2 = \{sno \rightarrow head\}$
 - *lossless*, because
 - $R_1 \cap R_2 = \{sno\}$, and is the key of R_1 and R_2
 - *non-dependency preservation*, because
 - $(F_1 \cup F_2)^+ \neq F^+, dept \rightarrow head$ is lost,

Example of Dependency Preserving (cont.)

- for $dept \rightarrow head$ in $\mathbf{F}$,

(1) with respect to R_1 ,

$$\begin{aligned} \mathbf{result} &= (dept \cap R_1)^+ \cap R_1 = \{dept\}^+ \cap \{sno, dept\} \\ &= \{dept, head\} \cap \{sno, dept\} \\ &= \{dept\} ; \end{aligned}$$

(2) with respect to R_2 ,

$$\begin{aligned} \mathbf{result} &= (dept \cap R_2)^+ \cap R_2 \\ &= \phi^+ \cap \{sno, head\} = \phi \end{aligned}$$

$dept \rightarrow head$ is not preserved

Example of Dependency Preserving (cont.)

■ Decomposition 2

$R_1(sno, dept), \quad F_1 = \{sno \rightarrow dept\}$

$R_2(dept, head), \quad F_2 = \{dept \rightarrow head\}$

■ *lossless-join*, because

■ $R_1 \cap R_2 = \{dept\}$, and is the key of R_2

■ *dependency preservation*, because

■ $(F_1 \cup F_2)^+ = F^+$

8.5 Algorithms for Decomposition

- BCNF decomposition
- 3NF decomposition

8.5.1 BCNF Decomposition

- A decomposition algorithm that gives *lossless* schema in BCNF is shown in Fig.8.12
 - F^+ is computed at first to determine whether or not α is the superkey of subschema R_i
 - if **other** methods can be used to determine whether or not α is the superkey of subschema R_i , then F^+ need not be computed
- An alternative easily-understood and straight-forward description of BCNF decomposition is given below

Given: R, F
holding on R

找出non-BCNF子模式 R_i , 进一步分解

 北京邮电大学
 R_i 中分解为2部分—公共属性 α , 其中 $\alpha \rightarrow \beta$ 组成单独的BCNF模式

result := { R };
done := false;
while (**not done**) **do**

if (there is a non-BCNF subschema R_i in **result**)
then begin

let $\alpha \rightarrow \beta$ be a nontrivial functional dependency that holds on R_i , such that

(1) α is not the **superkey** for R_i , i.e. $\alpha \rightarrow \beta$ is

not

R_i is non-BCNF
because of $\alpha \rightarrow \beta$

in F^+ , with respect to the restriction F_i of F to R_i !!!

(2) $\alpha \cap \beta = \emptyset$; /* $\alpha \rightarrow \beta$ 中无冗余属性*/

result := (**result** - R_i) $\cup$ {($R_i - \beta$)} $\cup$ {(α, β)};
end

α remains
in R_i

BCNF Decomposition (cont.)

■ Note

- to determine whether or not R_i is in BCNF, the restriction of F to R_i and the **candidate keys of R_i** should be computed !!!
- R_i is not in BCNF, because with respect to $\alpha \rightarrow \beta$ that holds on R_i , α is not the super key of R_i
- the algorithm replaces *non-BCNF* R_i with **$(R_i - \beta)$ and (α, β)**
 - *non-BCNF* R_i is decomposed into $(R_i - \beta)$ and BCNF (α, β)
- the restriction of F to the schema (α, β) is $\alpha \rightarrow \beta$ holds, α is the superkey for (α, β) , so (α, β) is in BCNF

Example 1

- $R = (A, B, C)$
 $F = \{A \rightarrow B$
 $B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF ($B \rightarrow C$ but B is not superkey)
- Decomposition
 - $R_1 = (B, C)$
 - $R_2 = (A, B)$

Example 2

- *class* (*course_id*, *title*, *dept_name*, *credits*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)
- Functional dependencies:
 - *course_id* → *title*, *dept_name*, *credits*
 - *building*, *room_number* → *capacity*
 - *course_id*, *sec_id*, *semester*, *year* → *building*, *room_number*, *time_slot_id*
- A candidate key {*course_id*, *sec_id*, *semester*, *year*}.
- BCNF Decomposition:
 - *course_id* → *title*, *dept_name*, *credits* holds
 - but *course_id* is not a superkey.
 - We replace *class* by:
 - *course*(*course_id*, *title*, *dept_name*, *credits*)
 - *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)

BCNF Decomposition (Cont.)

- *course* is in BCNF
 - How do we know this?
- *building, room_number* → *capacity* holds on *class-1*
 - but {*building, room_number*} is not a superkey for *class-1*.
 - We replace *class-1* by:
 - *classroom* (*building, room_number, capacity*)
 - *section* (*course_id, sec_id, semester, year, building, room_number, time_slot_id*)
- *classroom* and *section* are in BCNF.

BCNF and Dependency Preservation

It is not always possible to get a BCNF decomposition that is dependency preserving

- $R = (J, K, L)$

$$F = \{JK \rightarrow L$$

$$L \rightarrow K \}$$

Two candidate keys = JK and JL

- R is not in BCNF
- Any decomposition of R will fail to preserve

$$JK \rightarrow L$$

This implies that testing for $JK \rightarrow L$ requires a join

Example 4

BCNF Decomposition

- Considering the schema $R(C, T, H, R, S, G)$, and $F = \{CS \rightarrow G, C \rightarrow T, TH \rightarrow R, HR \rightarrow C, HS \rightarrow R\}$, give a decomposition of R into BCNF
- Step1. With respect to R and F , the unique candidate key is HS , and R is not in BCNF
- Step2. Initially, $result := \{R\} = \{CTHRSG\}$
 - R is not in BCNF, because there is $CS \rightarrow G$ in F , and CS is not the candidate key of R
 - $R(CTHRSG)$ is decomposed into $R_1(CSG)$ and $R_2(CTHR)$
 - $R_1(CSG)$ is in BCNF, $F_1 = \{CS \rightarrow G\}$

Example 4

BCNF Decomposition (cont.)

- Step3. With respect to $R_2(CTHRS)$
 - !!!the restriction of F to $R_2(CTHRS)$ is
 $F_2 = \{C \rightarrow T, TH \rightarrow R, HR \rightarrow C, HS \rightarrow R\}$
 - !!the candidate key is HS
 - $R_2(CTHRS)$ is not in BCNF, because there is $C \rightarrow T$ in F_2 , and C is not the candidate key of R_2
 - $R_2(CTHRS)$ is decomposed into $R_{21}(CT)$ and $R_{22}(CHRS)$
 - $R_{21}(CT)$ is in BCNF, $F_{21} = \{C \rightarrow T\}$

Example 4

BCNF Decomposition (cont.)

- Step4. With respect to $R_{22}(CHRS)$
 - !!the restriction of F to $R_{22}(CHRS)$ is
 $F_{22} = \{HR \rightarrow C, HS \rightarrow R, CH \rightarrow R\}$ /* $CH \rightarrow TH, TH \rightarrow R$
 - !!the candidate key is HS
 - $R_{22}(CHRS)$ is not in BCNF, because there is $HR \rightarrow C$ in F_{22} , and HR is not candidate key of R_{22}
 - $R_{22}(CHRS)$ is decomposed into $R_{221}(HRC)$ and $R_{222}(HRS)$
 - $R_{221}(HRC)$ is in BCNF
- Step5. With respect to $R_{222}(HRS)$,
 - the restriction of F to $R_{222}(HRS)$ is
 $F_{222} = \{HS \rightarrow R\}$, and the candidate key is HS
 - $R_{222}(HRS)$ is in BCNF

Example 4

BCNF Decomposition (cont.)

- Finally, the BCNF decomposition of $R(C, T, H, R, S, G)$ is
 - $R_1(CSG)$, $R_{21}(CT)$, $R_{221}(HRC)$, $R_{222}(HRS)$

8.5.2 3NF and Its Decomposition

Third Normal Form: Motivation

- There are some situations where
 - BCNF is not dependency preserving, and
 - efficient checking for FD violation on updates is important
- Solution: define a weaker normal form, called Third Normal Form (3NF)
 - Allows some redundancy (with resultant problems; we will see examples later)
 - But functional dependencies can be checked on individual relations without computing a join.
 - There is always a lossless-join, dependency-preserving decomposition into 3NF.

3NF Example

- Relation *dept_advisor*:

- *dept_advisor* (*s_ID*, *i_ID*, *dept_name*)

$$F = \{s_ID, dept_name \rightarrow i_ID, i_ID \rightarrow dept_name\}$$

- Two candidate keys: *s_ID*, *dept_name*, and *i_ID*, *s_ID*

- *R* is in 3NF

- *s_ID*, *dept_name* $\rightarrow$ *i_ID*

- *s_ID*, *dept_name* is a superkey

- *i_ID* $\rightarrow$ *dept_name*

- *dept_name* is contained in a candidate key

Redundancy in 3NF

- There is some redundancy in this schema
- Example of problems due to redundancy in 3NF
 - $R = (J, K, L)$
 $F = \{JK \rightarrow L, L \rightarrow K\}$

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
$null$	l_2	k_2

repetition of information (e.g., the relationship l_1, k_1)

- $(i_ID, dept_name)$

Testing for 3NF

- Optimization: Need to check only FDs in F , need not check all FDs in F^+ .
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
 - this test is rather more expensive, since it involve finding candidate keys
 - testing for 3NF has been shown to be NP-hard
 - Interestingly, decomposition into third normal form (described shortly) can be done in polynomial time

3NF Decomposition

- A decomposition algorithm that gives lossless and dependency preserving schemas in 3NF is shown in Fig.?.13
- Note:
 - **all** candidate keys should be founded out
 - **only one** F_c should be computed *at first*

0. Find out *all* candidate keys for R

1. Find out *a* canonical cover F_c for F ;

求最简FD:
去除掉传递依
赖和冗余属性

$i := 0$; /* R_0 is null*/

2. **for each** functional dependency $\alpha \rightarrow \beta$ in F_c **do**

if *none of the schemas R_j , $1 \leq j \leq i$ contains $\alpha \beta$*

then begin

$i := i + 1$;

$R_i := \alpha \beta$

对 F_c 中每一个未包括在已有子模式中的函数依赖，构造一个BCNF/3NF子模式

end

3. **if** none of the schemas R_j , $1 \leq j \leq i$ contains a candidate key for R

then begin

$i := i + 1$;

$R_i :=$ any candidate key for R ;

对 R 的候选键单独构造一个关系模式

end

4. **return** ($R_1, R_2, \dots, R_i$)

Example 1

3NF Decomposition

- Consider schema $R(X, Y, Z, W)$, and $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$ that holds on R . Give the lossless, dependency-preserving decomposition of this schema into 3NF

The solution is as follows:

- Step1. $\{YW\}^+ = R$, $\{YW\}$ is the unique candidate key
- Step2. With respect to $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$,
 - considering X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$,
 $\{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ implies F , so X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$ are all extraneous attributes, and one of **canonical covers** for F is
$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$$

Example 1

3NF Decomposition (cont.)

- the other *canonical covers* for F are
$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\}, \text{ or}$$
$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\}, \text{ or}$$
$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$
- Step3. As far as $F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ is concerned,
 - on the basis of 3NF decomposition algorithm, the subschema responding to each functional dependency in F_c are
$$R_1 = \{XZ\}, R_2 = \{YZ\}, R_3 = \{XW\}$$
 - for the candidate key $\{YW\}$, $R_4 = \{YW\}$ is generated, so *one of* the 3NF decomposition is
$$\{XZ, YZ, XW, YW\}$$

Example 1

3NF Decomposition (cont.)

- With respect to the other *canonical covers*

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$

and the candidate key $\{YW\}$,

the corresponding 3NF decomposition are

$$\{XZ, YX, WX, YW\}$$

$$\{XZ, YX, WZ, YW\}$$

$$\{XZ, YZ, WZ, YW\}$$

Example 2

3NF Decomposition (cont.)

- Assume there is a schema **S** (**Sno**, **Cno**, **Tname**, **Taddr**, **Grade**).
 - each student has only one grade for each course he/she took;
 - each course is taught by only one teacher;
 - each teacher has only one address. (Suppose duplication of name is not allowed for teachers)
- 1. Write down three instances of functional dependency of the schema S mentioned above.
 - $(\text{Sno}, \text{Cno}) \rightarrow \text{Grade}$
 - $\text{Cno} \rightarrow \text{Tname}$
 - $\text{Tname} \rightarrow \text{Taddr}$

Example 2

3NF Decomposition (cont.)

- 2. Write down the candidate keys of schema S.

Sno, Cno

由于 $\{Sno, Cno\}^+ = (Sno, Cno, Tname, Taddr, Grade)$

- 3. Decompose the schema S into the third normal form.

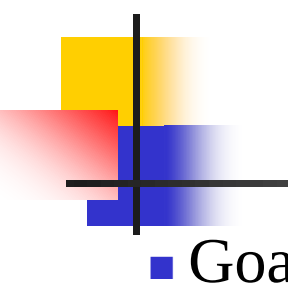
S1(Sno, Cno, Grade)

S2(Cno, Tname)

S3(Tname, Taddr)

8.5.4 Comparison of BCNF and 3NF

- It is always possible to decompose a relation into a set of relations that are in 3NF such that:
 - the decomposition is lossless
 - the dependencies are preserved
- It is always possible to decompose a relation into a set of relations that are in BCNF such that:
 - the decomposition is lossless
 - it may not be possible to preserve dependencies.



Design Goals

- Goal for a relational database design is:
 - BCNF.
 - Lossless join.
 - Dependency preservation.
- If we cannot achieve this, we accept one of
 - Lack of dependency preservation
 - Redundancy due to use of 3NF

Computing of Candidate Keys

- 定理。如果属性A只出现在F中函数依赖的左部, 则A一定是主属性, 必然出现在候选键中
- 输入: 关系模式R及其函数依赖集F
- 输出: R的所有候选关键字
- 方法:
 - step1.将R的所有属性分为四类:
 - L类: 仅出现在F中函数依赖左部的属性
 - R类: 仅出现在F中函数依赖右部的属性
 - N类: 在F中函数依赖左右两边均未出现的属性
 - LR类: 在F中函数依赖左右两边均出现的属性

Computing of Candidate Keys (cont.)

- 并令 X_set 代表L、N类， Y_set 代表LR类；
- step2. 求 X_set^+ ，若包含了R的所有属性，则 X_set 即为R的唯一候选关键字，转step5；
 否则转step3
- step3. 在 Y_set 中取一属性A，求 $(X_set \cup A)^+$ ，若它包含了R的所有属性，则转step4；
 否则，调换一属性反复进行这一过程，直到试完所有 Y_set 中的属性
- step4. 如果已找出所有的候选关键字，则转step5；
 否则，在 Y_set 中依此取两个、三个，...，求他们的属性闭包，直到其闭包包含R的所有的属性
- step5. 停止，输出结果

Computing of Candidate Keys (cont.)

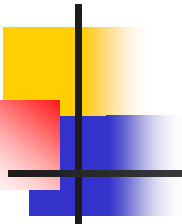
- E.g.1. $R(X, Y, Z, W)$, and $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$
 - L类: Y, W
 - R、N: 空
 - LR: X, Z
 - $X_set = \{Y, W\}$
 - $Y_set = \{X, Z\}$
 - $(YW)^+ = R$, so YW is the candidate key

Computing of Candidate Keys (cont.)

- E.g.2. $R(B, D, I, O, Q, S)$, and

$$F = \{B \rightarrow Q, I \rightarrow S, IS \rightarrow Q, S \rightarrow D\}$$

- L类: B, I
- R类: D, Q
- N类: O
- LR类: S
- $X_set = \{B, I, O\}$
- $Y_set = \{S\}$
- $(BIO)^+ = R$, so BIO is a candidate key



$R(A,B,C,D,E)$

$F=\{AB \rightarrow CE, E \rightarrow AB, C \rightarrow D\}$

无L类属性

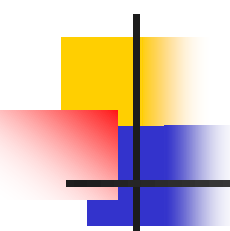
键: AB, E

Appendix B 本章习题类型

- 给定关系表 $r(R)$ 和若干函数依赖 $\alpha \rightarrow \beta$, 判断 r 是否满足 $\alpha \rightarrow \beta$
- 判断关于函数依赖的一些公式是否成立
 - 例如, 如果 $A \rightarrow B, B \rightarrow C$, 则 $A \rightarrow C$
(根据Amstron公理系统)
- 根据文字描述, 抽象出函数依赖关系
- 求候选键(算法, 附录D)
- 计算属性闭包 α^+
- 计算函数依赖集 F 的最小正则集 F_c

Appendix G 本章习题类型 (续)

- 给定关系模式 R 和定义在 R 上的函数依赖集 F ，判断 R 属于第几范式，为什么？
 - 只考虑1NF, 2NF, 3NF, BCNF
- 判断一个模式分解是否为无损连接、函数依赖保持
- 给定非3NF的关系模式 R 和定义在 R 上的函数依赖集 F ，将 R 分解为第三范式
- 给定非BCNF的关系模式 R 和定义在 R 上的函数依赖集 F ，将 R 分解为BCNF范式



Have a break
